

Notebook as World Model

EE290/194 Final Report

Group 4

May 15, 2026

Abstract

We introduce NOTEVOLVE, a training-free agentic framework that uses a Jupyter notebook as both the execution environment and the persistent world model for an LLM agent. Unlike program-centric approaches such as AlphaEvolve, which evolve a single program artifact, NOTEVOLVE evolves an entire *notebook state*—comprising code cells, markdown annotations, execution outputs, and live kernel memory—across iterative rounds. This design unifies LLM-accessible memory (text summaries, prior code, structured plans) with non-LLM memory (kernel variables, cached data structures, intermediate computations) in a single, executable, multimodal document. We demonstrate competitive or superior performance to AlphaEvolve on open mathematical optimization problems, and achieve 100% pass rate on some Terminal Bench tasks using a locally-served Nemotron-3-Nano-30B model, substantially outperforming the same model without the notebook harness. [\[Hengyu: need to add new experiment results\]](#)

1 Introduction

Large language models (LLMs) have demonstrated strong, and in some domains superhuman, capabilities in reasoning, coding, and scientific problem solving. Turning these capabilities into reliable agents, however, remains difficult on complex multi-step tasks: the agent must plan, execute code, inspect intermediate results, preserve useful state, recover from errors, and decide which partial solutions deserve further computation. Prior work has made substantial progress on individual pieces of this problem. Tool-using and code-acting agents interleave reasoning with external actions [16, 14, 15]; memory-based agents retain feedback, skills, or long-term context across interactions [11, 13, 8]; and evolutionary or test-time search systems improve candidate programs through repeated generation and evaluation [9, 6, 10, 4, 1].

Despite this progress, memory, execution, and evolution are often treated as separate design choices. Tool-use systems may execute commands without a durable representation of the evolving workspace; memory systems may store textual summaries without preserving executable computational state; and evolutionary systems typically optimize isolated programs or search policies rather than a full record of experimentation. This separation is especially limiting for scientific and machine-learning workloads, where progress depends not only on the final code artifact but also on intermediate data, plots, diagnostics, failed attempts, and live variables that shape later decisions.

Our goal is to design an agentic system in which memory, execution environment, and evolution state are unified. We propose NOTEVOLVE, a training-free agent framework that uses a **Jupyter notebook as the agent’s world model**. The notebook is not merely a user interface: it is the central state object through which the agent perceives, acts, and remembers. It stores text, code, outputs, plots, tables, and execution history in a structured document, while the live kernel preserves non-text state such as Python objects, loaded datasets, fitted models, cached intermediate results, and imported libraries.

This design gives the agent a persistent workspace that is both human-readable and executable. Unlike program-evolution systems that maintain a population of isolated code artifacts, NOTEVOLVE

evolves a full notebook trajectory: helper functions, failed attempts, diagnostics, visualizations, and live variables all remain available for later rounds. Unlike text-memory agents, the notebook can re-execute and inspect prior work rather than merely summarize it. The resulting state can serve simultaneously as memory for the LLM, an execution substrate for tools, and the object being refined by test-time search.

Our evaluation studies this claim across mathematical optimization, Terminal-Bench tasks [5], and MLE-bench-style machine-learning engineering [2]. In mathematical optimization, NOTEVOLVE matches or exceeds AlphaEvolve/OpenEvolve-style baselines on the problems tested while using only a small number of notebook sessions. On Terminal-Bench, the same local Nemotron backend improves substantially when placed inside the notebook harness, showing that persistent state and structured execution can compensate for weaker base-model capability. On MLE-bench tasks, the results further suggest that the choice of memory substrate and orchestration policy is as important as the choice of LLM backend.

2 Related Work

Tool-using and code-acting agents. ReAct [16] established the now-standard pattern of interleaving reasoning, tool actions, and observations. Later systems extend this idea with richer action spaces: CodeAct [14] lets agents act through executable code, and SWE-agent [15] shows that the agent-computer interface can strongly affect software-engineering performance. NOTEVOLVE follows this line of work, but shifts the interface from isolated commands or scripts to a persistent notebook whose cells can be edited, executed, inspected, and reused across rounds. This makes the interface itself part of the agent’s problem representation: instead of treating execution as a transient side effect, the notebook records the evolving state of the task in a form that is both executable and legible to the model.

Agent memory. Long-horizon agents need memory beyond the current prompt. Reflexion [11] stores verbal feedback from prior attempts, Voyager [13] accumulates reusable skills, and MemGPT [8] frames context management as a memory hierarchy. These systems mostly store text or discrete artifacts. NOTEVOLVE instead uses the notebook as a mixed memory substrate: prompt-visible cells provide LLM-readable memory, while the live kernel preserves computational memory that need not fit in the context window. In this sense, our memory mechanism is not only a retrieval layer but an executable external state, allowing later decisions to condition on prior computations, intermediate datasets, plots, and failed attempts without re-deriving them from scratch.

Program evolution and discovery. FunSearch [9], AlphaEvolve [6], and OpenEvolve [10] demonstrate that LLM-generated programs can be improved through automated evaluation and selection. More recent systems such as EvoX [4] and AdaEvolve [1] further study how the search process itself should adapt over time, for example by evolving search strategies or allocating compute according to improvement signals. These methods motivate our view of agentic work as a test-time optimization process, but their evolving object is typically a candidate solution, program, or search policy. NOTEVOLVE generalizes this view from program evolution to state evolution: the object being improved is an entire executable research workspace, including code, analysis, outputs, intermediate variables, and failed attempts.

3 Proposed Approach: the Notebook as World Model

The central design choice in NOTEVOLVE is to treat the Jupyter notebook not as a passive transcript, but as the agent’s *world model*: the state that is observed, acted on, scored, selected, and evolved. This choice is natural because a notebook combines three roles that are usually separated across different agent systems.

First, a notebook is an evolvable state. Program-evolution systems such as AlphaEvolve [6] evolve code artifacts and store scored programs in an evolutionary database. NOTEVOLVE generalizes the artifact being evolved from a program to a full notebook state: code cells, markdown notes, execution outputs, plots, diagnostics, and round summaries. A branch is therefore not just a candidate program; it is an entire research trajectory that can contain multiple hypotheses, helper functions, failed attempts, and the current best construction. The outer loop can select, clone, merge, or perturb these notebook states in the same spirit that evolutionary systems select programs.

Second, a notebook is a multimodal memory management system. Pure text memory, such as conversation history or a markdown scratchpad, is limited to what can be sent back to the LLM. A notebook provides a dual-layer memory:

- **LLM memory** (text-based): Cell source code, markdown summaries, execution output strings, and structured round-end summaries—all visible to the LLM in the notebook rendering.
- **Non-LLM memory** (kernel state): Python variables, loaded datasets, trained models, cached intermediate results, and imported libraries persist in the Jupyter kernel across rounds. This memory can be reused without being serialized into the LLM context.

Notebook cells also support multiple modalities—code, Markdown/L^AT_EX, tables, plots, images, and rich display objects—which makes the notebook a more faithful state representation for scientific and data-centric workflows than a plain text log.

Third, a notebook is an executable environment. Unlike a static memory document, each code cell can be executed and re-executed against a persistent kernel. The agent can define a helper in one cell, use it in later cells, revise it, and re-run stale downstream computations. This makes previous work operationally reusable: expensive data loading, search state, cached tables, and trained models can remain alive in kernel memory while the LLM only sees compact summaries of them.

3.1 Framework Architecture

NOTEVOLVE is organized as an outer–inner loop. The **outer loop** maintains a population of notebook branches, where $\mathcal{N}_t^{(i)}$ denotes the i -th notebook branch at round t and $s_t^{(i)}$ denotes its score. After each round, every branch produces an updated notebook state together with logs and evaluator scores. An evolution algorithm then maps the scored branch set to the branch set for the next round. This evolution algorithm is deliberately modular: it can be as simple as best-of- n selection, tournament selection, or MAP-Elites-style archive updates. The key abstraction is that the evolutionary unit is a scored notebook state, not a single function or file.

The **inner loop** is one notebook-agent round executed independently on each branch. It has three phases:

- **Planning / beginning**: the current notebook state is rendered into a compact text context; the LLM receives the task, scaffold prompt, notebook snapshot, and available tool schemas; its plan

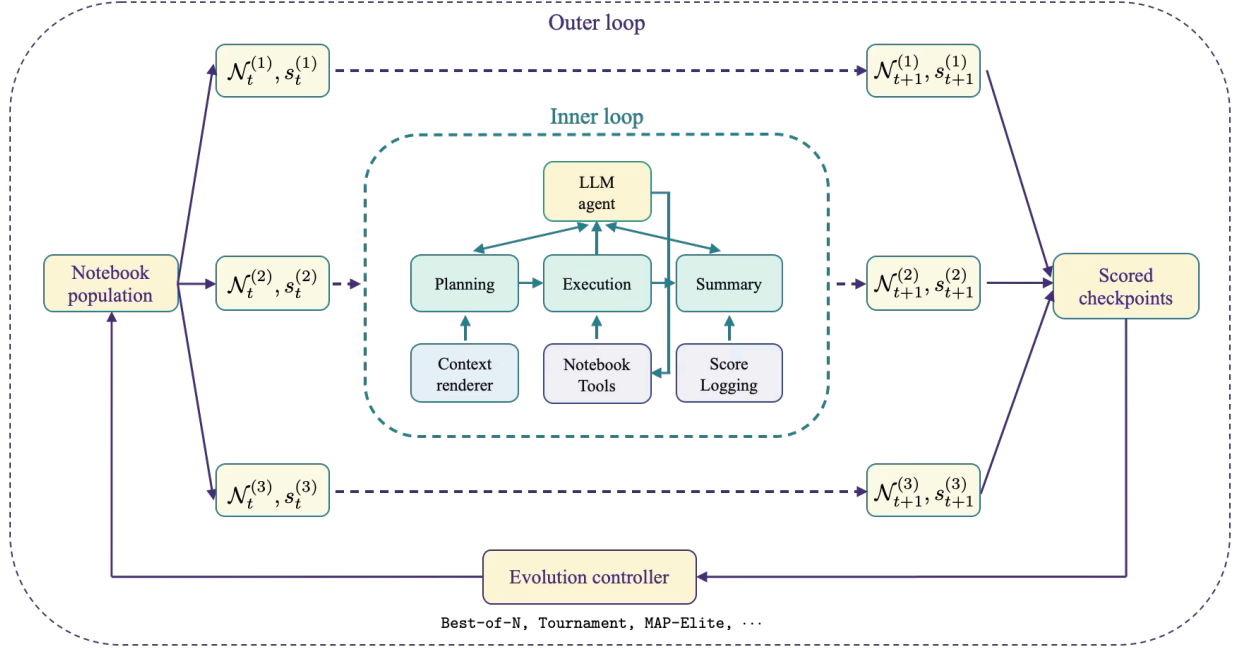


Figure 1: NOTEVOLVE outer-inner architecture. The outer loop maintains a population of notebook branches. At round t , branch i is represented by a full notebook state $\mathcal{N}_t^{(i)}$ and score $s_t^{(i)}$. The **Evolution Controller** fans this population out into branch-specific notebook states, runs a notebook-agent round on each branch, collects updated notebook states $\mathcal{N}_{t+1}^{(i)}$ with scores $s_{t+1}^{(i)}$, and then selects or evolves the next population. This outer update can instantiate simple best-of- n selection, tournament selection, or MAP-Elites-style archive updates. The inner loop is expanded for one branch: **Context_renderer** renders the notebook into compact context, the **LLM Agent** acts through the **Notebook Tool Pool** and **Jupyter Kernel**, the **Evaluator** records scores, and the **Cleaner** summarizes and cleans the notebook before checkpointing the next state.

is saved back into the notebook.

- **Execution:** the LLM calls notebook tools to add, edit, run, read, fold, unfold, and summarize cells. Code execution goes through a live Jupyter kernel, while evaluator tools log real scores that can later drive the outer evolution loop.
- **Summary / cleanup:** the LLM emits cleanup instructions; the system folds or unfolds cells, deletes obsolete current-round cells, inserts a round summary, and saves a checkpoint.

Three components interact throughout the inner loop: the **Notebook State** as context, the **LLM Agent**, and the **Notebook Tool Pool**. The notebook state is rendered for perception; the LLM decides actions; the tool pool mutates the notebook, executes code, manages context, and reports outputs and scores. Figure 1 illustrates the branch-level architecture, while Algorithm 1 gives the corresponding outer-inner loop pseudocode. In the pseudocode, C_{sys} denotes the fixed system context formed by the task description, scaffold prompt, and tool-use prompt/tool schemas.

Algorithm 1 NOTEVOLVE notebook evolution framework pseudocode.

```

1 Input: seed population  $\mathcal{P}_0 = \{(\mathcal{N}_0^{(i)}, s_0^{(i)})\}_{i=1}^{m_0}$ , round budget  $T$ , max tool calls  $n$ .
2  $C_{\text{sys}} \leftarrow$  task description + scaffold prompt + tool-use prompt/tool schemas.
3 for  $t = 0, \dots, T - 1$  do
4   for each  $(\mathcal{N}_t^{(i)}, s_t^{(i)}) \in \mathcal{P}_t$  do
5     Planning / beginning:
6      $C_t^{(i)} \leftarrow \text{Context\_renderer}(\mathcal{N}_t^{(i)}) + C_{\text{sys}}$  render notebooks into compact context
7      $(\mathcal{N}_t^{(i)}, C_t^{(i)}) \leftarrow \text{Planner}(\mathcal{N}_t^{(i)}, C_t^{(i)}, \text{LLM})$  make plans for execution
8     Execution:
9     for  $k = 1, \dots, n$  do
10       $\tau_{t,k}^{(i)} \leftarrow \text{Propose\_tool}(C_t^{(i)}, \text{LLM})$  propose one tool call
11       $(\mathcal{N}_t^{(i)}, C_t^{(i)}) \leftarrow \text{Call\_tool}(\tau_{t,k}^{(i)}, \mathcal{N}_t^{(i)}, C_t^{(i)})$ 
12      Typical tools: add/edit/run/read/summarize/fold/unfold cells.
13      Summary / cleanup:
14       $\mathcal{N}_{t+1}^{(i)} \leftarrow \text{Cleaner}(\mathcal{N}_t^{(i)}, C_t^{(i)}, \text{LLM})$  summarize; delete/fold/unfold cells
15       $s_{t+1}^{(i)} \leftarrow \text{Evaluator}(\mathcal{N}_{t+1}^{(i)})$  log scores
16       $\mathcal{S}_{t+1} \leftarrow \{(\mathcal{N}_{t+1}^{(i)}, s_{t+1}^{(i)})\}_{i=1}^{m_t}$ 
17       $\mathcal{P}_{t+1} \leftarrow \text{Evolution\_alg}(\mathcal{S}_{t+1})$  next population
18 return  $\arg \max_{(\mathcal{N}, s) \in \bigcup_{u=1}^T \mathcal{S}_u} s$ .
```

3.2 Context Management: Rendering, Folding, and Cleanup

A raw Jupyter notebook is a JSON document containing cell identifiers, execution counters, metadata, MIME bundles, and full outputs. Sending this file directly to an LLM wastes context on serialization details and makes long-running notebook histories quickly unusable. NOTEVOLVE therefore introduces a renderer and several context-management tools that make the notebook usable as LLM memory.

Rendering to compact text. At the start of a round, the notebook is converted into a structured text view. Each cell is shown with its index, type, execution state, short description, and output summary. Recent or explicitly unfolded cells show full source and clipped outputs; older or explicitly folded cells are shown as one-line summaries. This preserves the notebook’s sequential structure while removing JSON boilerplate.

Context tools during execution. The tool pool includes output summarization, cell folding/unfolding, single-cell reading, and full-output expansion tools: `summarize_cell`, `fold_cell`, `unfold_cell`, `read_cell`, and `expand_output`. After each execution, the agent must call `summarize_cell` to convert raw output into a short semantic memory. Long outputs are clipped in tool results and rendered contexts, with explicit pointers to `expand_output` when full details are needed. This lets the agent preserve the information value of an experiment without repeatedly paying for every printed line or traceback.

Round-end compression. At the end of each round, the agent emits cleanup instructions. Obsolete or superseded cells from the current round may be deleted; successful but no-longer-active experiments can be folded; cells that define the current best method or important state can remain unfolded. A structured round summary is inserted as a markdown cell. Thus context compaction is not a one-time preprocessing step, but a continual part of the agent’s interaction protocol.

Figure 2 illustrates this context lifecycle across the beginning, execution, and final cleanup phases.

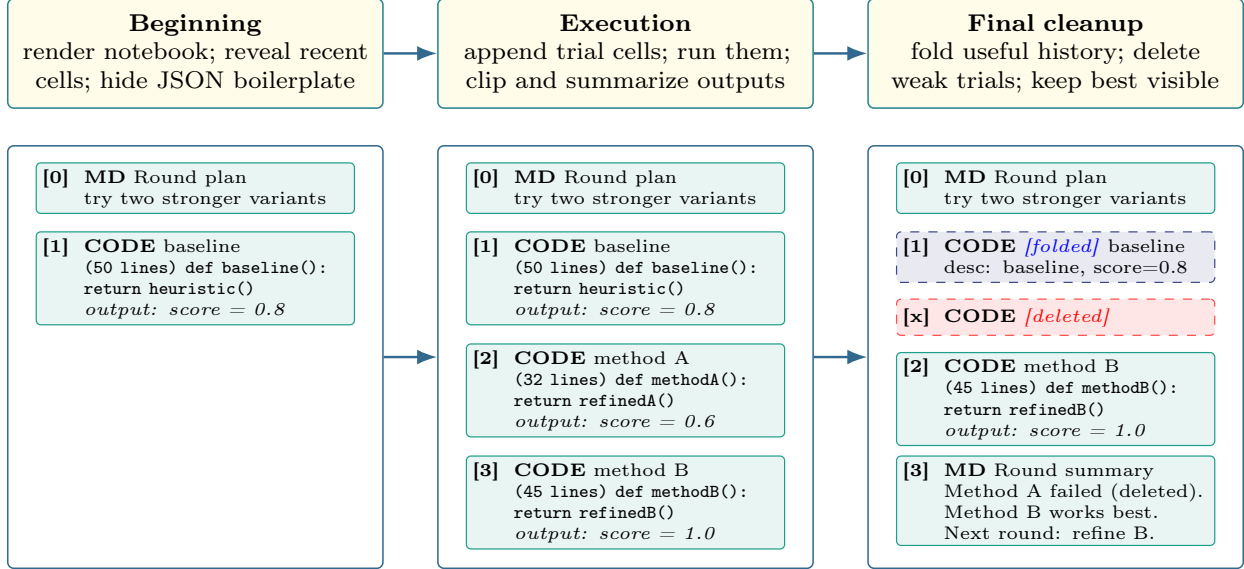


Figure 2: Context management in NOTEVOLVE shown as a two-row, three-phase pipeline. The top row names the context operation performed in each phase; the bottom row shows a concrete rendered-notebook example. Green cells are unfolded and contribute source code plus clipped output to the LLM context. In the execution phase, the agent appends two unfolded trial cells: method A scores worse than the baseline, while method B improves the score. During final cleanup, the baseline is folded into a short description, the low-scoring trial is deleted from the next-round context, and the best trial remains unfolded so the next round can directly inspect and build on it.

4 Results

We evaluate NOTEVOLVE on a few complementary benchmarks: (1) mathematical optimization problems from the AlphaEvolve suite, testing iterative refinement over many rounds, and (2) Terminal Bench tasks, testing single-shot task completion with a locally-served open-weight model, and (3) MLEBench, which tests performance on Kaggle challenges, gauging real-world multi-step coding ability.

4.1 Mathematical Optimization (AlphaEvolve-type Problems)

We compare NOTEVOLVE against the published AlphaEvolve results [6] and OpenEvolve¹, an open-source evolutionary coding agent inspired by AlphaEvolve. All OpenEvolve and NOTEVOLVE runs in this comparison use Gemini 3 Flash as the base model. The benchmark suite contains four mathematical optimization problems where the objective is directly evaluable: circle packing, the Erdős minimum-overlap problem, the Heilbronn triangle problem, and the 3D min-max distance-distribution problem.

For these experiments, NOTEVOLVE uses a lightweight multi-branch inspiration algorithm. We maintain four long-lived notebook branches with independent kernels. After each branch round, the branch writes a scored notebook checkpoint to a shared snapshot database. Before the next round, a branch receives a compact, code-only inspiration block sampled from the best sibling-branch checkpoints. Thus the unit of evolution is a full notebook state, while cross-branch information flow

¹<https://github.com/algorithmicsuperintelligence/openevolve>

Table 1: Mathematical optimization results with Gemini 3 Flash. $\uparrow$ means higher is better and $\downarrow$ means lower is better. **Bold** marks the best or tied-best score. For NOTEVOLVE, we create 4 branches of jupyter books and set the number of rounds to be 4 with 25 maximal tool calls each round. For OpenEvolve, we set the number of evolutionary iterations to be 100 with a maximal program population size being 60.

Problem	AlphaEvolve	OpenEvolve	NOTEVOLVE
Circle Packing $\uparrow$	2.635	2.4672	2.635983
Erdős Min-Overlap $\downarrow$	0.380923	0.4334	0.38089
Heilbronn Triangle $\uparrow$	0.03653	0.0349	0.03653
Min-Max Distrib. $\uparrow$	0.24004706	0.2300	0.24005088

is controlled through short code excerpts rather than copying entire notebooks. Appendix A gives the detailed pseudocode.

With the same base model, NOTEVOLVE obtains the best score on three of the four problems and ties AlphaEvolve on Heilbronn, while OpenEvolve trails on all four objective values. These results suggest that evolving notebook states can be competitive with program-only evolutionary systems on objective-driven mathematical search.

4.1.1 Why Does the Notebook Help?

To understand why NOTEVOLVE performs well, we inspect a high-performing Circle Packing² branch notebook. Figure 3 shows that the notebook is more than a code transcript: it stores *plots of the current packing*, *persistent variables* such as `best_c`, `best_r`, and `best_s`, reusable solver functions, clipped outputs, and round summaries. These artifacts let later cells diagnose geometric gaps, warm-start from the best known layout, and avoid repeating failed directions.

In this branch, the solver evolves from grid-search baselines to a penalty formulation with LP-based radius evaluation, then to shake/refine and gap-relocation operators, and finally to SLSQP refinement plus inflation-relaxation polishing. The x-axis in Figure 3 is therefore indexed by round and cell rather than by method name: the solver is an evolving sequence of notebook states.

4.2 Terminal Bench (Pass/Fail Agent Tasks)

To evaluate NOTEVOLVE on practical software engineering tasks, we adapt 10 tasks from Terminal Bench [5] and run them with a locally-served NVIDIA Nemotron-3-Nano-30B model (3.5B active parameters via Mixture-of-Experts) served through vLLM. This tests whether the notebook harness can compensate for a smaller, less capable model on tasks that require multi-step reasoning, tool use, and error recovery. Table 2 compares NOTEVOLVE (single trial) against the bare Nemotron model called directly via the Terminal Bench harness (8 trials per task).

The bare Nemotron model achieves only 33.8% pass rate across 80 trials, failing entirely on 5 out of 9 tested tasks. With the NOTEVOLVE harness, the *same model* achieves 100% pass rate on all 11 tasks in a single trial each. This indicates both the capability of weaker models to leverage the the notebook harness effectively, and that weaker model performance can be augmented at test-time on coding tasks by utilizing our notebook-based approach while oftentimes *lowering wall-clock runtime*.

²Circle Packing asks the agent to place 26 non-overlapping circles inside the unit square and maximize the sum of their radii. A valid score is therefore the total radius sum after checking boundary and overlap constraints; the AlphaEvolve-level target used in this experiment is 2.635.

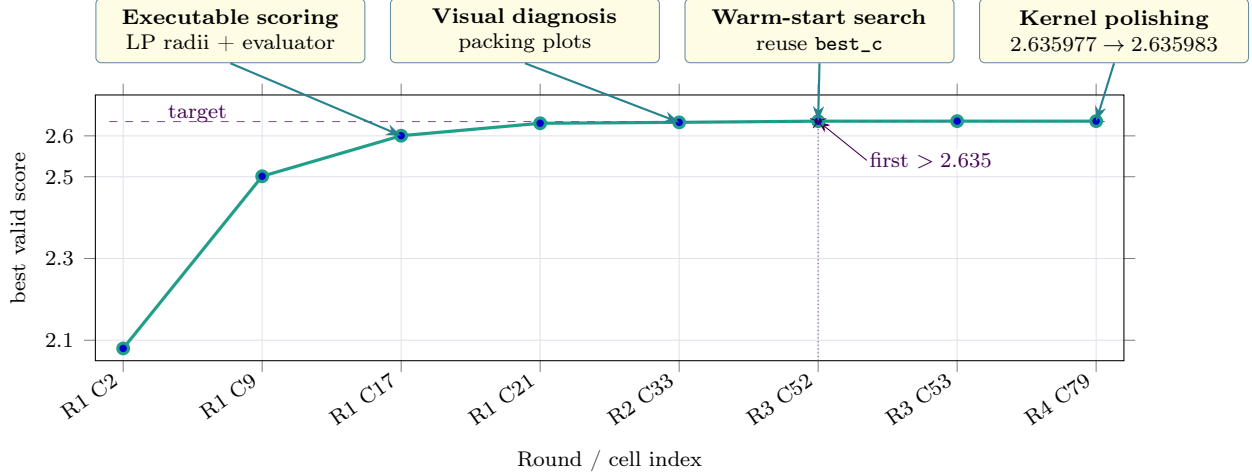


Figure 3: Qualitative mechanism observed in the Circle Packing notebook branch. The x-axis indexes selected notebook rounds and cells where the best valid score improved; solver names are described in the main text rather than used as tick labels. The dashed line marks the AlphaEvolve-level target of 2.635, and the star marks the first cell that exceeds it. The notebook accumulates more than source code: it stores solver definitions, validated best variables such as `best_c`, visualization outputs, failed attempts, and round summaries. Later cells reuse this state as warm starts for increasingly specialized search operators, culminating in a small but measurable final improvement from kernel-state polishing.

Table 2: Terminal Bench results: NOTEVOLVE (single trial) vs. bare Nemotron-3-Nano-30B (8 trials). Time in seconds.

Task	Bare Nemotron		Ours (Notebook + Nemotron)	
	Pass Rate	Avg Time	Pass Rate	Time
hello-world	8/8	41s	1/1	12s
csv-to-parquet	8/8	156s	1/1	17s
fix-permissions	8/8	45s	1/1	25s
extract-safely	2/8	55s	1/1	40s
simple-web-scraper	1/8	163s	1/1	51s
download-youtube	0/8	311s	1/1	149s
vim-terminal-task	0/8	303s	1/1	151s
count-dataset-tokens	0/8	358s	1/1	254s
oom	0/8	250s	1/1	26s
train-fasttext	0/8	639s	1/1	82s

4.3 MLEBench

We next evaluate NOTEVOLVE on MLEBench [2], a benchmark of real-world machine-learning engineering tasks drawn from public Kaggle competitions. These tasks require agents to inspect data, design and debug training pipelines, run experiments, and submit predictions under task-specific metrics. We run all methods with the same locally hosted Nemotron-120B-Super model [7], served on 8×H100 GPUs, and compare four harnesses: NOTEVOLVE (NotEvolve), AIDE with population size one [3], CheetahClaw [12], and OpenEvolve [10].

This comparison is intended to probe how different methods to retain memory over test-time optimization affect long-horizon coding performance. NotEvolve stores intermediate development

state in a Jupyter notebook: previous cells, execution outputs, kernel state, diagnostics, and partial results remain available as the agent iterates. AIDE is the closest evolutionary baseline, but represents state as a single evolving code file; setting its population size to one makes the comparison closer to NotEvolve while removing cross-individual search effects. CheetahClaw follows a Claude-Code-style development loop in which the agent edits a directory of files over time, providing a strong baseline for realistic software-engineering workflows. OpenEvolve instead maintains a population of separately evolving code files, allowing multiple solution islands to develop in parallel and exchange progress through selection.

Table 5 shows that NOTEVOLVE is competitive with CheetahClaw, the strongest realistic code-development harness in our comparison. While CheetahClaw is often much faster, NOTEVOLVE consistently produces solutions near the best task score and is best or tied-best on several tasks, including **random-acts-of-pizza**, **tps-dec-2021**, and **dog-breed**. However, NOTEVOLVE generally uses the largest token budget and has high wall-clock time. The wall-clock cost is dominated by CPU-side execution rather than LLM querying, which we empirically found was due to notebook agent proposing more complex or compute-intensive pipelines. The larger token budget, however, does reflect the cost of exposing rich notebook state to the model, and motivates future work on more sophisticated context compaction for notebook representations.

4.4 KernelBench

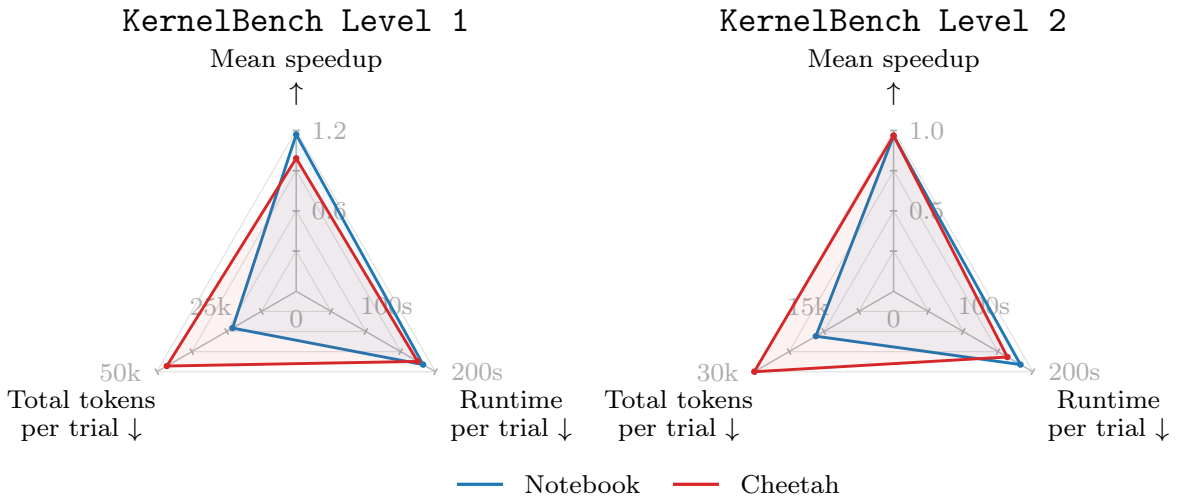


Figure 4: Radar plots for KernelBench Level 1 and Level 2 over three seeds. Each axis is linear and starts at zero, with the outer radius set to the largest observed value within that level and quantity. Polygons use means only. Total tokens are computed as input plus output tokens per trial. For speedup, higher is better; for tokens and runtime, lower is better, so lower-cost methods appear closer to the center on those axes.

5 Conclusion and Discussion

We proposed NOTEVOLVE, a training-free agentic framework that uses a Jupyter notebook as a unified state for memory, execution, and evolution. By evolving the full notebook state—code cells, markdown plans, outputs, visualizations, summaries, and live kernel variables—NOTEVOLVE gives the LLM both an executable workspace and a persistent world model. NOTEVOLVE matches or exceeds

Table 3: Performance, token usage, and runtime across Kaggle tasks. Each task is grouped into metric score, tokens consumed, and runtime. Runtimes are reported as mean $\pm$ standard deviation. Best values are bolded within each row; for metric rows, best follows the metric direction, while for tokens and runtime lower is better. Superscripts 1/2/3 correspond to the original gold/silver/bronze markers.

Nemotron					
Task	Measure	NotEvolve	AIDE	CheetahHarness	OpenEvolve
random-acts-of-pizza	AUC $\uparrow$	² 0.780 (0.769$\pm$0.009)	0.643 (0.626 $\pm$ 0.024)	² 0.774 (0.767 $\pm$ 0.009)	0.531 (0.526 $\pm$ 0.007)
	Tokens	3.1M	152k	1.7M	37k
	Runtime	1h18m $\pm$ 2h12m	50m37s $\pm$ 1h12m	2m04s $\pm$ 1m10s	58m38s $\pm$ 1h14m
nomad2018	RMSLE $\downarrow$	² 0.061 (0.062 $\pm$ 0.000)	² 0.062 (0.065 $\pm$ 0.003)	² 0.060 (0.061$\pm$0.000)	0.259 (0.259 $\pm$ 0.000)
	Tokens	2.5M	302k	1.0M	25k
	Runtime	1h19m $\pm$ 2h21m	1h20m $\pm$ 3h14m	1m29s $\pm$ 1m49s	10m43s $\pm$ 7m41s
spaceship-titanic	acc $\uparrow$	³ 0.814 (0.814 $\pm$ 0.000)	0.802 (0.795 $\pm$ 0.006)	¹ 0.822 (0.814$\pm$0.013)	0.802 (0.802 $\pm$ 0.000)
	Tokens	1.7M	137k	819k	83k
	Runtime	1h22m $\pm$ 1h33m	44m13s $\pm$ 56m11s	1m35s $\pm$ 1m05s	8m24s $\pm$ 3m03s
spooky-author	LL $\downarrow$	0.416 (0.426 $\pm$ 0.013)	0.549 (0.577 $\pm$ 0.035)	0.463 (0.496 $\pm$ 0.028)	0.363 (0.363$\pm$0.000)
	Tokens	5.0M	120k	307k	45k
	Runtime	1h23m $\pm$ 2h01m	26m10s $\pm$ 46m50s	1m04s $\pm$ 16s	9m47s $\pm$ 2m37s
jigsaw-toxic	AUC $\uparrow$	0.962 (0.962 $\pm$ 0.000)	0.975 (0.975$\pm$0.000)	0.973 (0.684 $\pm$ 0.277)	—
	Tokens	1.3M	88k	914k	—
	Runtime	45m19s $\pm$ 2m40s	1h19m $\pm$ 2h28m	2m45s $\pm$ 1m31s	—
tps-dec-2021	acc $\uparrow$	0.938 (0.938$\pm$0.000)	0.923 (0.923 $\pm$ 0.000)	0.938 (0.897$\pm$0.039)	0.908 (0.888 $\pm$ 0.029)
	Tokens	1.2M	33k	1.9M	72k
	Runtime	2h33m $\pm$ 3h09m	22m49s $\pm$ 28m07s	5m02s $\pm$ 2m16s	3h55m $\pm$ 5h22m
tps-may-2022	acc $\uparrow$	0.927 (0.927 $\pm$ 0.000)	0.886 (0.876 $\pm$ 0.015)	0.928 (0.904 $\pm$ 0.027)	0.982 (0.979$\pm$0.004)
	Tokens	8.6M	59k	2.5M	46k
	Runtime	51m28s $\pm$ 44m23s	1h18m $\pm$ 44m21s	5m07s $\pm$ 4m50s	6m40s $\pm$ 1m48s
dog-breed	LL $\downarrow$	0.778 (1.162$\pm$0.483)	2.499 (2.499 $\pm$ 0.000)	4.178 (5.010 $\pm$ 1.332)	4.814 (4.814 $\pm$ 0.000)
	Tokens	2.5M	91k	1.1M	100k
	Runtime	14m24s $\pm$ 0s	38m37s $\pm$ 35m11s	6m27s $\pm$ 2m33s	8m14s $\pm$ 34s
nyc-taxi (5.3GB)	RMSE $\downarrow$	4.459 (5.245 $\pm$ 0.714)	5.815 (6.316 $\pm$ 0.709)	4.684 (7.325 $\pm$ 3.735)	3.119 (3.185$\pm$0.092)
	Tokens	3.5M	137k	955k	68k
	Runtime	17m42s $\pm$ 0s	50m29s $\pm$ 41m39s	6m22s $\pm$ 2m57s	6m56s $\pm$ 5s
dogs-vs-cats (1.2GB)	LL $\downarrow$	0.354 (0.472 $\pm$ 0.131)	0.121 (0.402$\pm$0.262)	0.665 (0.774 $\pm$ 0.155)	0.628 (0.628 $\pm$ 0.000)
	Tokens	4.8M	109k	1.6M	39k
	Runtime	26m28s $\pm$ 5m18s	1h52m $\pm$ 1h14m	9m56s $\pm$ 6m24s	3m47s $\pm$ 2m01s

AlphaEvolve/OpenEvolve-style baselines on open mathematical optimization problems, substantially improves the open-weight Nemotron-3-Nano-30B model on Terminal Bench, and remains competitive

Table 4: KernelBench results over three seeds.

Metric	Level 1 (50 × 3 = 150)	Level 2 (50 × 3 = 150)	Overall (300 trials)
Speedup			
Notebook mean ± std	1.170 ± 1.340	0.966 ± 0.455	1.096 ± 1.107
Cheetah mean ± std	0.992 ± 0.499	0.969 ± 0.263	0.982 ± 0.417
Notebook wins > 1.0×	42	34	76
Cheetah wins > 1.0×	22	27	49
Notebook wins ≥ 1.5×	17	3	20
Cheetah wins ≥ 1.5×	9	3	12
Notebook wins ≥ 2.0×	13	2	15
Cheetah wins ≥ 2.0×	4	0	4
Tokens per trial			
Notebook input mean ± std	20,029 ± 15,393	13,599 ± 8,747	16,814 ± 12,906
Cheetah input mean ± std	44,577 ± 50,405	28,139 ± 21,512	36,386 ± 39,597
Notebook output mean ± std	2,803 ± 1,224	3,143 ± 709	2,973 ± 1,013
Cheetah output mean ± std	1,882 ± 486	1,935 ± 299	1,908 ± 404
Wall-clock per trial (seconds)			
Notebook mean ± std	182.2 ± 0.0	182.2 ± 0.0	182.2 ± 0.0
Cheetah mean ± std	174.4 ± 16.9	163.4 ± 20.4	168.9 ± 19.5

on realistic machine-learning engineering tasks. These results suggest that for long-horizon agentic work, the state representation itself is a central part of agent capability.

Future work should focus on making notebook-state evolution more efficient, robust, and broadly applicable. The main practical limitation is token usage: notebooks can accumulate large histories, so better state distillation, cell folding policies, output summarization, and cleanup strategies are needed to preserve useful information while reducing context cost. Another direction is improving kernel-state reliability through stronger checkpointing, replay, dependency tracking, and sandboxing. Finally, NOTEVOLVE should be extended to broader domains such as kernel/system optimization, compiler and runtime tuning, simulation-driven design, and larger scientific or machine-learning pipelines, where iterative experimentation and reusable computational state are essential.

References

- [1] Mert Cemri, Shubham Agrawal, Akshat Gupta, Shu Liu, Audrey Cheng, Qiuyang Mang, Ashwin Naren, Lutfi Eren Erdogan, Koushik Sen, Matei Zaharia, Alex Dimakis, and Ion Stoica. Adaevolve: Adaptive llm driven zeroth-order optimization. *arXiv preprint arXiv:2602.20133*, 2026.
- [2] Jun Shern Chan, Neil Chowdhury, Oliver Jaffe, James Aung, Dane Sherburn, Evan Mays, Giulio Starace, Kevin Liu, Leon Maksin, Tejal Patwardhan, Lilian Weng, and Aleksander Mądry. Mle-bench: Evaluating machine learning agents on machine learning engineering. *arXiv preprint arXiv:2410.07095*, 2024.
- [3] Zhengyao Jiang, Dominik Schmidt, Dhruv Srikanth, Dixing Xu, Ian Kaplan, Deniss Jacenko, and Yuxiang Wu. Aide: Ai-driven exploration in the space of code. *arXiv preprint arXiv:2502.13138*, 2025.
- [4] Shu Liu, Shubham Agarwal, Monishwaran Maheswaran, Mert Cemri, Zhifei Li, Qiuyang Mang, Ashwin Naren, Ethan Boneh, Audrey Cheng, Melissa Z. Pan, Alexander Du, Kurt Keutzer, Alvin Cheung, Alexandros G. Dimakis, Koushik Sen, Matei Zaharia, and Ion Stoica. Evox: Meta-evolution for automated discovery. *arXiv preprint arXiv:2602.23413*, 2026.
- [5] Mike A. Merrill, Alexander G. Shaw, Nicholas Carlini, Boxuan Li, Harsh Raj, Ivan Bercovich, Lin Shi, Jeong Yeon Shin, Thomas Walshe, E. Kelly Buchanan, Junhong Shen, Guanghao Ye, Haowei Lin, et al. Terminal-bench: Benchmarking agents on hard, realistic tasks in command line interfaces. *arXiv preprint arXiv:2601.11868*, 2026.
- [6] Alexander Novikov, Ngân Vũ, Marvin Eisenberger, Emilien Dupont, Po-Sen Huang, Adam Zsolt Wagner, Sergey Shirobokov, Borislav Kozlovskii, Francisco J. R. Ruiz, Abbas Mehrabian, M. Pawan Kumar, Abigail See, Swarat Chaudhuri, George Holland, Alex Davies, Sebastian Nowozin, Pushmeet Kohli, and Matej Balog. Alphaevolve: A coding agent for scientific and algorithmic discovery. *arXiv preprint arXiv:2506.13131*, 2025.
- [7] NVIDIA, :, Aakshita Chandiramani, Aaron Blakeman, Abdullahi Olaoye, Abhibha Gupta, Abhilash Somasamudramath, Abhinav Khattar, Adeola Adesoba, Adi Renduchintala, Adil Asif, Aditya Agrawal, Aditya Vavre, Ahmad Kiswani, Aishwarya Padmakumar, Ajay Hotchandani, Akanksha Shukla, Akhiad Bercovich, Aleksander Ficek, Aleksandr Shaposhnikov, Alex Gronskiy, Alex Kondratenko, Alex Neefus, Alex Steiner, Alex Yang, Alexander Bukharin, Alexander Young, Ali Hatamizadeh, Ali Taghibakhshi, Alina Galiautdinova, Alisa Liu, Alok Kumar, Ameya Sunil Mahabaleshwarkar, Amir Klein, Amit Zuker, Amnon Geifman, Anahita Bhiwandiwalla, Ananth Subramaniam, Andrew Tao, Anjaney Shrivastava, Anjulie Agrusa, Ankur Srivastava, Ankur Verma, Ann Guan, Anna Shors, Annamalai Chockalingam, Anubhav Mandarwal, Aparnaa Ramani, Arham Mehta, Arti Jain, Arun Venkatesan, Asha Anoosheh, Ashwath Aithal, Ashwin Poojary, Asif Ahamed, Asit Mishra, Asli Sabanci Demiroz, Asma Kuriparambil Thekkumpate, Atefeh Sohrabizadeh, Avinash Kaur, Ayush Dattagupta, Barath Subramaniam Anandan, Bardiya Sadeghi, Barnaby Simkin, Ben Lanir, Benedikt Schifferer, Benjamin Chislett, Besmira Nushi, Bilal Kartal, Bill Thiede, Bitu Darvish Rouhani, Bobby Chen, Boris Ginsburg, Brandon Norick, Branislav Kisacanin, Brian Yu, Bryan Catanzaro, Buvanewari Mani, Carlo del Mundo, Chankyu Lee, Chanran Kim, Chantal Hwang, Chao Ni, Charles Wang, Charlie Truong, Cheng-Ping Hsieh, Chenhan Yu, Chenjie Luo, Cherie Wang, Chetan Mungekar, Chintan Patel, Chris Alexiuk, Chris Holguin, Chris Wing, Christian Munley, Christopher Parisien, Chuck Desai, Chunyang

Sheng, Collin Neale, Cyril Meurillon, Dakshi Kumar, Dan Gil, Dan Su, Dane Corneil, Daniel Afrimi, Daniel Burkhardt Eliuth Triana, Daniel Egert, Daniel Fatade, Daniel Lo, Daniel Rohrer, Daniel Serebrenik, Daniil Sorokin, Daria Gitman, Daria Levy, Darko Stosic, David Edelsohn, David Messina, David Mosallanezhad, David Tamok, Deena Donia, Deepak Narayanan, Devin O’Kelly, Dheeraj Peri, Dhruv Nathawani, Di Wu, Dima Rekesh, Dina Yared, Divyanshu Kakwani, Dmitry Konyagin Brandon Tuttle, Dong Ahn, Dongfu Jiang, Dorrin Poorkay, Douglas O’Flaherty, Duncan Riach, Dusan Stosic, Dustin Van Stee, Edgar Minasyan, Edward Lin, Eileen Peters Long, Elad Segal, Elena Lantz, Elena Lewis, Ellie Evans, Elliott Ning, Eric Chung, Eric Harper, Eric Pham-Hung, Eric W. Tramel, Erick Galinkin, Erik Pounds, Esti Etrog, Evan Briones, Evan Wu, Evelina Bakhturina, Evgeny Tsykunov, Ewa Dobrowolska, Farshad Saberi Movahed, Farzan Memarian, Fay Wang, Fei Jia, Felipe Soares, Felipe Vieira Frujeri, Feng Chen, Fengguang Lin, Ferenc Galko, Fortuna Zhang, Frankie Siino, Frida Hou, Gantavya Bhatt, Gargi Prasad, Geethapriya Venkataramani, Geetika Gupta, George Armstrong, Gerald Shen, Giulio Borghesi, Gordana Neskovic, Gorkem Batmaz, Grace Lam, Grace Wu, Greg Pauloski, Greyson Davis, Grigor Nalbandyan, Guoming Zhang, Guy Farber, Guyue Huang, Haifeng Qian, Haran Kumar Shiv Kumar, Harry Kim, Harsh Sharma, Hayate Iso, Hayley Ross, Herbert Hum, Herman Sahota, Hexin Wang, Himanshu Soni, Hiren Upadhyay, Huy Nguyen, Iain Cunningham, Ido Galil, Ido Shahaf, Iginio Padovani, Igor Gitman, Igor Shovkun, Ikroop Dhillon, Ilya Loshchilov, Ingrid Kelly, Itamar Schen, Itay Levy, Ivan Moshkov, Izik Golan, Izzy Putterman, Jain Tu, Jan Baczek, Jan Kautz, Jane Polak Scowcroft, Janica Rosenberg, Jared Casper, Jarrod Pflum, Jason Grant, Jason Sewall, Jatin Mitra, Jeffrey Glick, Jenny Chen, Jesse Oliver, Jiacheng Xu, Jiafan Zhu, Jialin Song, Jian Zhang, Jiaqi Zeng, Jie Lou, Jill Milton, Jim Chow, Jimmy Zhang, Jinhang Choi, Jining Huang, Jocelyn Huang, Joel Caruso, Joey Conway, Joey Guman, Johan Jatko, John Kamalu, Johnny Greco, Jonathan Cohen, Jonathan Raiman, Joseph Jennings, Joyjit Daw, Juan Yu, Julio Tapia, Junkeun Yi, Jupinder Parmar, Jyothi Achar, Kari Briski, Kartik Mattoo, Katherine Cheung, Katherine Luna, Keith Wyss, Kevin Shih, Kezhi Kong, Khanh Nguyen, Khushi Bhardwaj, Kirill Buryak, Kirthi Shankar Sivamani, Konstantinos Krommydas, Kris Murphy, Krishna C. Puvvada, Krzysztof Pawelec, Kumar Anik, Laikh Tewari, Laya Sleiman, Leo Du, Leon Derczynski, Li Ding, Lilach Ilan, Lingjie Wu, Lizzie Wei, Luis Vega, Lun Su, Maarten Van Segbroeck, Maer Rodrigues de Melo, Magaret Zhang, Mahan Fathi, Makesh Narsimhan Sreedhar, Makesh Sreedhar, Makesh Tarun Chandran, Manuel Reyes Gomez, Maor Ashkenazi, Marc Cuevas, Marc Romeijn, Margaret Zhang, Mark Cai, Mark Gabel, Markus Kliegl, Martyna Patelka, Maryam Moosaei, Matthew Varacalli, Matvei Novikov, Mauricio Ferrato, Mehrzad Samadi, Melissa Corpuz, Meng Xin, Mengdi Wang, Mengru Wang, Meredith Price, Micah Schaffer, Michael Andersch, Michael Boone, Michael Evans, Michael Z Wang, Miguel Martinez, Mikail Khona, Mike Chrzanowski, Mike Hollinger, Mingyuan Ma, Minseok Lee, Mohammad Dabbah, Mohammad Shoeybi, Mostofa Patwary, Nabin Mulepati, Nader Khalil, Najeeb Nabwani, Nancy Agarwal, Nanthini Balasubramaniam, Narimane Hennouni, Narsi Kodukula, Natalie Hereth, Nathaniel Pinckney, Nave Assaf, Negar Habibi, Nestor Qin, Neta Zmora, Netanel Haber, Nick Reamaroon, Nickson Quak, Nidhi Bhatia, Nikhil Jukar, Nikki Pope, Nikolai Ludwig, Nima Tajbakhsh, Nir Ailon, Nirmal Juluru, Nirmalya De, Nowel Pitt, Oleg Rybakov, Aleksii Hrinchuk, Aleksii Kuchaiev, Olivier Delalleau, Oluwatobi Olabiyi, Omer Ullman Argov, Omri Almog, Omri Puny, Oren Tropp, Otavio Padovani, Ouye Xie, Parth Chadha, Pasha Shamis, Paul Gibbons, Pavlo Molchanov, Peter Belcak, Peter Jin, Pinky Xu, Piotr Januszewski, Pooya Jannaty, Prachi Shevate, Pradeep Thalasta, Pranav Prashant Thombre, Prasoon Varshney, Prerana Gambhir, Pritam Gundecha, Przemek Tredak, Qing Miao, Qiyu Wan, Quan Tran Minh, Rabeeh Karimi Mahabadi, Rachel Oberman, Rachit Garg, Rahul Kandu, Raina Zhong, Ran El-Yaniv, Ran Zilberstein, Rasoul Shafipour, Renee Yao, Renjie

- Pi, Richard Mazzaresse, Richard Wang, Rick Izzo, Ridhima Singla, Rima Shahbazy, Rishabh Garg, Ritika Borkar, Ritu Gala, Riyad Islam, Robert Clark, Robert Hesse, Roger Waleffe, Rohit Varma Kalidindi, Rohit Watve, Roi Koren, Ron Fan, Ruchika Kharwar, Ruisi Cai, Ruoxi Zhang, Russell J. Hewett, Ryan Prenger, Ryan Timbrook, Ryota Egashira, Sadegh Mahdavi, Sagar Singh Ashutosh Joshi, Sahil Modi, Samuel Krizan, Sandeep Pombra, Sanjay Kariyappa, Sanjeev Satheesh, Santiago Pombo, Saori Kaji, Satish Pasumarthi, Saurav Mishra, Saurav Muralidharan, Scott Hara, Sean Narenthiran, Sebastian Rogawski, Seonjin Na, Seonmyeong Bak, Sepehr Sameni, Seth Poulos, Shahar Mor, Shantanu Acharya, Shaona Ghosh Adam Lord, Sharath Turuvekere Sreenivas, Shaun Kotek, Shaya Gharghabi, Shelby Thomas, Sheng-Chieh Lin, Shibani Likhite, Shiqing Fan, Shiyang Chen, Shreya Gopal, Shrimai Prabhume, Shubham Pachori, Shubham Toshniwal, Shuo Zhang, Shuoyang Ding, Shyam Renjith, Shyamala Prayaga, Siddhartha Jain, Simeng Sun, Sirisha Rella, Sirshak Das, Smita Ithape, Sneha Harishchandra S, Somshubra Majumdar, Soumye Singhal, Sri Harsha Singudasu, Sriharsha Niverty, Stas Sergienko, Stefana Gloginic, Stefania Alborghetti, Stephen Ge, Stephen McCullough, Sugam Dipak Devare, Suguna Varshini Velury, Sukrit Rao, Sumeet Kumar Barua, Sunny Gai, Suseella Panguluri, Sushil Koundinyan, Swathi Patnam, Sweta Priyadarshi, Swetha Bhendigeri, Syeda Nahida Akter, Sylendran Arunagiri, Tailling Yuan, Talor Abramovich, Tan Bui, Tan Yu, Terry Kong, Thanh Do, Thomas Gburek, Thorgane Marques, Tiffany Moore, Tijmen Blankevoort, Tim Moon, Timothy Ma, Tiya Mitra, Tomasz Grzegorzec, Tomer Asida, Tomer Bar Natan, Tomer Keren, Tomer Ronen, Traian Rebedea, Trenton Starkey, Tugrul Konuk, Twinkle Vashishth, Tyler Condensa, Udi Karpas, Ushnish De, Vahid Noorozi, Vahid Noroozi, Vanshil Atul Shah, Veena Vaidyanathan, Venkat Srinivasan, Venmugil Elango, Victor Cui, Vijay Korthikanti, Vikas Mehta, Virginia Adams, Virginia Wu, Vitaly Kurin, Vitaly Lavrukhin, Vladimir Anisimov, Wan Seo, Wanli Jiang, Wasi Uddin Ahmad, Wei Du, Wei Ping, Wei-Ming Chen, Wendy Quan, Wenliang Dai, Wenwen Gao, Will Jennings, William Zhang, Xiaowei Ren, Xiaowen Xin, Xin Li, Yang Yu, Yangyi Chen, Yaniv Galron, Yashaswi Karnati, Yejin Choi, Yev Meyer, Yi-Fu Wu, Yian Zhang, Ying Lin, Yonatan Geifman, Yonggan Fu, Yoshi Suhara, Youngeun Kwon, Yuan Zhang, Yuki Huang, Zach Moshe, Zhilin Wang, Zhiyu Cheng, Zhongbo Zhu, Zhuolin Yang, Zihan Liu, Zijia Chen, Zijie Yan, and Zuhair Ahmed. Nemotron 3 super: Open, efficient mixture-of-experts hybrid mamba-transformer model for agentic reasoning, 2026.
- [8] Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G. Patil, Ion Stoica, and Joseph E. Gonzalez. Memgpt: Towards llms as operating systems. *arXiv preprint arXiv:2310.08560*, 2023.
- [9] Bernardino Romera-Paredes, Mohammadamin Barekatain, Alexander Novikov, Matej Balog, M. Pawan Kumar, Emilien Dupont, Francisco J. R. Ruiz, Jordan S. Ellenberg, Pengming Wang, Omar Fawzi, Pushmeet Kohli, and Alhussein Fawzi. Mathematical discoveries from program search with large language models. *Nature*, 625:468–475, 2024.
- [10] Asankhaya Sharma. Openevolve: an open-source evolutionary coding agent, 2025.
- [11] Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. *arXiv preprint arXiv:2303.11366*, 2023.
- [12] CheetahClaws Team. Cheetahclaws: An extensible, python-native agent system for autonomous multi-model workflows. *github*, 2026.

- [13] Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Voyager: An open-ended embodied agent with large language models. *arXiv preprint arXiv:2305.16291*, 2023.
- [14] Xingyao Wang, Yangyi Chen, Lifan Yuan, Yizhe Zhang, Yunzhu Li, Hao Peng, and Heng Ji. Executable code actions elicit better llm agents. *arXiv preprint arXiv:2402.01030*, 2024.
- [15] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. Swe-agent: Agent-computer interfaces enable automated software engineering. *arXiv preprint arXiv:2405.15793*, 2024.
- [16] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models. *arXiv preprint arXiv:2210.03629*, 2022.

A Experimental Evolution Algorithm

Algorithm 2 Multi-branch inspiration evolution used in the mathematical optimization experiments

```

1 Input: seed notebook  $\mathcal{N}_0$ , branch count  $m$ , generations  $G$ , rounds per generation  $r$ , inspirations
   per round  $K$ .
2 Initialize branches  $\{\mathcal{B}^{(i)}\}_{i=1}^m$  as copies of  $\mathcal{N}_0$  with independent persistent kernels.
3 Initialize shared snapshot database  $\mathcal{D} \leftarrow \emptyset$ .
4 for  $g = 1, \dots, G$  do
5   for branch  $i = 1, \dots, m$  do
6      $\mathcal{I}_g^{(i)} \leftarrow \text{SampleInspirations}(\mathcal{D}, i, K)$ 
7      $C_{\text{insp}}^{(i)} \leftarrow \text{RenderCodeOnly}(\mathcal{I}_g^{(i)})$  no outputs; clipped code cells
8      $(\mathcal{N}_g^{(i)}, s_g^{(i)}, h_g^{(i)}) \leftarrow \text{InnerLoop}(\mathcal{B}^{(i)}, C_{\text{insp}}^{(i)}, r)$ 
9      $\mathcal{D} \leftarrow \mathcal{D} \cup \{(\mathcal{N}_g^{(i)}, s_g^{(i)}, h_g^{(i)}, i, g)\}$ 
10    if token budget exceeded then  $\mathcal{B}^{(i)} \leftarrow \text{Rebirth}(\text{OwnBest}(\mathcal{D}, i))$ 
11    if tournament is enabled at generation  $g$  then
12       $i_{\text{lose}} \leftarrow \arg \min_i \text{BestScore}(\mathcal{D}, i)$  among eligible branches
13       $\mathcal{N}_{\text{donor}} \leftarrow \text{SampleTopQuartile}(\mathcal{D} \setminus i_{\text{lose}})$ 
14       $\mathcal{B}^{(i_{\text{lose}})} \leftarrow \text{Rebirth}(\mathcal{N}_{\text{donor}})$ 
15 return  $\arg \max_{(\mathcal{N}, s) \in \mathcal{D}} s$  and the final snapshot database  $\mathcal{D}$ .
```

A.1 GLM MLEBench Results

Table 5: Performance, token usage, and runtime across Kaggle tasks. Each task is grouped into metric score, tokens consumed, and runtime. Runtimes are reported as mean $\pm$ standard deviation. Best values are bolded within each row; for metric rows, best follows the metric direction, while for tokens and runtime lower is better. Superscripts 1/2/3 correspond to the original gold/silver/bronze markers.

GLM				
Task	Measure	NA-GLM	AIDE-GLM	CH-GLM
random-acts-of-pizza	AUC $\uparrow$	0.650 (0.645 $\pm$ 0.005)	0.649 (0.609 $\pm$ 0.045)	0.672 (0.636$\pm$0.031)
	Tokens	1.8M	131k	230k
	Runtime	4m28s $\pm$ 0s	20m19s $\pm$ 9m45s	2m50s $\pm$ 35s
nomad2018	RMSLE $\downarrow$	³ 0.063 (0.065 $\pm$ 0.002)	² 0.061 (0.064$\pm$0.004)	² 0.062 (0.063 $\pm$ 0.001)
	Tokens	742k	122k	465k
	Runtime	42m10s $\pm$ 0s	16m16s $\pm$ 24m43s	11m18s $\pm$ 5m27s
spaceship-titanic	acc $\uparrow$	0.808 (0.799 $\pm$ 0.009)	¹ 0.822 (0.807$\pm$0.016)	³ 0.813 (0.807 $\pm$ 0.005)
	Tokens	945k	46k	320k
	Runtime	17m05s $\pm$ 0s	25m08s $\pm$ 17m38s	7m49s $\pm$ 4m43s
spooky-author	LL $\downarrow$	0.577 (0.609 $\pm$ 0.044)	0.576 (0.779 $\pm$ 0.300)	0.573 (0.621$\pm$0.041)
	Tokens	745k	35k	489k
	Runtime	20m37s $\pm$ 0s	16m51s $\pm$ 9m27s	12m47s $\pm$ 4m07s
jigsaw-toxic	AUC $\uparrow$	0.968 (0.957 $\pm$ 0.019)	0.969 (0.967 $\pm$ 0.003)	0.971 (0.952$\pm$0.017)
	Tokens	661k	43k	149k
	Runtime	22m45s $\pm$ 0s	46m22s $\pm$ 32m46s	11m41s $\pm$ 8m17s
tps-dec-2021	acc $\uparrow$	0.909 (0.906 $\pm$ 0.003)	0.942 (0.942$\pm$0.000)	0.902 (0.892 $\pm$ 0.013)
	Tokens	679k	12k	595k
	Runtime	32m30s $\pm$ 0s	17m50s $\pm$ 7m09s	18m33s $\pm$ 1m38s
tps-may-2022	acc $\uparrow$	0.830 (0.766 $\pm$ 0.111)	0.923 (0.923$\pm$0.000)	0.821 (0.723 $\pm$ 0.134)
	Tokens	902k	20k	605k
	Runtime	54m12s $\pm$ 0s	27m17s $\pm$ 16m15s	9m28s $\pm$ 4m34s
dog-breed	LL $\downarrow$	4.650 (4.650 $\pm$ 0.000)	4.469 (4.609$\pm$0.199)	4.749 (4.866 $\pm$ 0.166)
	Tokens	1.2M	62k	741k
	Runtime	—	1h18m $\pm$ 48m08s	20m02s $\pm$ 6m39s
nyc-taxi (5.3GB)	RMSE $\downarrow$	5.684 (5.684)	5.553 (5.553)	4.917 (7.296$\pm$2.592)
	Tokens	1.1M	20k	333k
	Runtime	—	48m05s $\pm$ 11m03s	11m23s $\pm$ 2m30s